PROJECT REPORT

ON

Feedo: A Real-Time Food Delivery Ecosystem

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

MASTER OF COMPUTER APPLICATIONS

(Computer Science and Technology)

JAN - MAY, 2026

SUBMITTED TO

Dr. Kapil

Assistant Professor

Dept. of Computer Science & Technology

Gurugram University

SUBMITTED BY

Ayush Kumar Mishra (241000170002)

Deepshikha Verma (241000170017)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GURUGRAM UNIVERSITY, GURUGRAM

DEPARTMENT OF COMPUTER SCIENCE &TECHNOLOGY
GURUGRAM UNIVERSITY , GURUGRAM

CERTIFICATE

This is to clarify that the work contained in the PROJECT entitled **" Feedo: A Real-Time Food Delivery Ecosystem "** , submitted by **Ayush Kumar Mishra (**241000170002**), Deepshikha verma** (241000170017) for the award of the **Master of Computer Applications (Computer Science and Technology)** to the Department of Engineering and Technology , Gurugram University , Gurugram , Haryana , is a record of bonafide work carried out by him under my direct supervision and guidance . I considered that the PROJECT has reached the standards and fulfilling the requirements of the rules and regulations relating to the nature of the degree . The contents embodied in the PROJECT have not been submitted for the award of any other degree or diploma in this or any other university .

Date :

Place : Gurugram, Haryana

Supervisor Signature :

Dr. Kapil
Assistant Professor
Department of Computer Science & Technology
Gurugram University

DEPARTMENT OF COMPUTER SCIENCE &TECHNOLOGY
GURUGRAM UNIVERSITY , GURUGRAM

CANDIDATE'S DECLARATION

I certify that

a. The work contained in the project is original and has been done by myself under the supervision of my supervisor.

b. The work has not been submitted to any other Institute for any degree or diploma.

c. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

d. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the PROJECT and giving their details in the references.

e. Whenever I have quoted written materials from other sources and due credit is given to the sources by citing them.

f. From the plagiarism test, it is found that the similarity index of whole PROJECT within 25% and single paper is less than 10 % as per the university guidelines.

Date :

Place : Gurugram, Haryana

Student Signature :

Ayush Kumar Mishra (241000170002)
Deepshikha Verma (241000170017)

DEPARTMENT OF COMPUTER SCIENCE &TECHNOLOGY

GURUGRAM UNIVERSITY , GURUGRAM

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Dr. Kapil** , for their continuous support, valuable guidance, and encouragement throughout the development of this project. Their insightful suggestions and constructive feedback helped me to complete this work successfully.

I am extremely thankful to the Chairperson and all the faculty members of the Department of Engineering and Technology, Gurugram University, Gurugram, for providing the necessary facilities and academic support.

I would also like to thank my friends and classmates for their cooperation, motivation, and helpful discussions during the course of this project.

Date :

Place : Gurugram, Haryana

Student Signature :

Ayush Kumar Mishra (241000170002)

Deepshikha Verma (241000170017)

ABSTRACT

Feedo is a comprehensive, state-of-the-art food delivery mobile application designed to bridge the gap between customers, restaurants, and delivery partners through real-time communication and robust logistics management. In the contemporary urban landscape, where the demand for efficient and transparent services has grown exponentially, Feedo leverages a modern technology stack—featuring a React Native cross-platform mobile frontend, a modular NestJS backend, and a flexible MongoDB data store—to provide a seamless and intuitive user experience. Key innovations include the integration of Mapbox SDK for high-fidelity driver tracking and Razorpay for secure, frictionless financial transactions, all orchestrated through an event-driven architecture using WebSockets for instantaneous updates. The project encompasses a rigorous system analysis phase, including detailed functional requirements and data flow diagrams that outline the secure transmission of information across the platform. Furthermore, the database architecture is meticulously designed to handle high concurrency, ensuring that restaurant menus and order histories are retrieved with minimal latency. Quality assurance was a primary focus, involving multi-tiered testing strategies—from unit testing core business logic to system-level performance evaluations—verifying the platform's stability under peak operational loads. This project demonstrates the successful implementation of a scalable ecosystem that prioritizes performance, security, and operational transparency, establishing a robust framework for the evolving on-demand service industry and significantly enhancing stakeholder satisfaction through real-time data visibility and reliable service delivery.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

1 Introduction

1.1 Introduction

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

1.2 Problem Statement

undefined

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

undefined

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

1.3 Scope of Research

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

1.4 Research Hypothesis

undefined

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

undefined

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

1.5 Objectives

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets

are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

1.6 Organization of the Report

undefined

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The FoodieExpress project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

undefined

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The FoodieExpress project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

2 Literature Review

2.1 Background

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

undefined

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

undefined

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

2.2 Summary of Literature Review and Research Gap

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

undefined

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned

driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

undefined

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

3 System Analysis and Requirements

3.1 Requirements Gathering

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

3.2 Feasibility Study

undefined

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

undefined

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

3.3 Functional Requirements

Table 3.1: Functional Requirements Analysis matrix detailing priority and impact

ID	Requirement	Module	Priority
FR1	Secure JWT Authentication	Auth	High
FR2	Real-time Order Status	Tracking	High
FR3	Multi-cart Management	Order	Medium

undefined

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

undefined

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

3.4 Non-Functional Requirements

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations

between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

undefined

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

undefined

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the

platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

4 System Design

4.1 Architecture

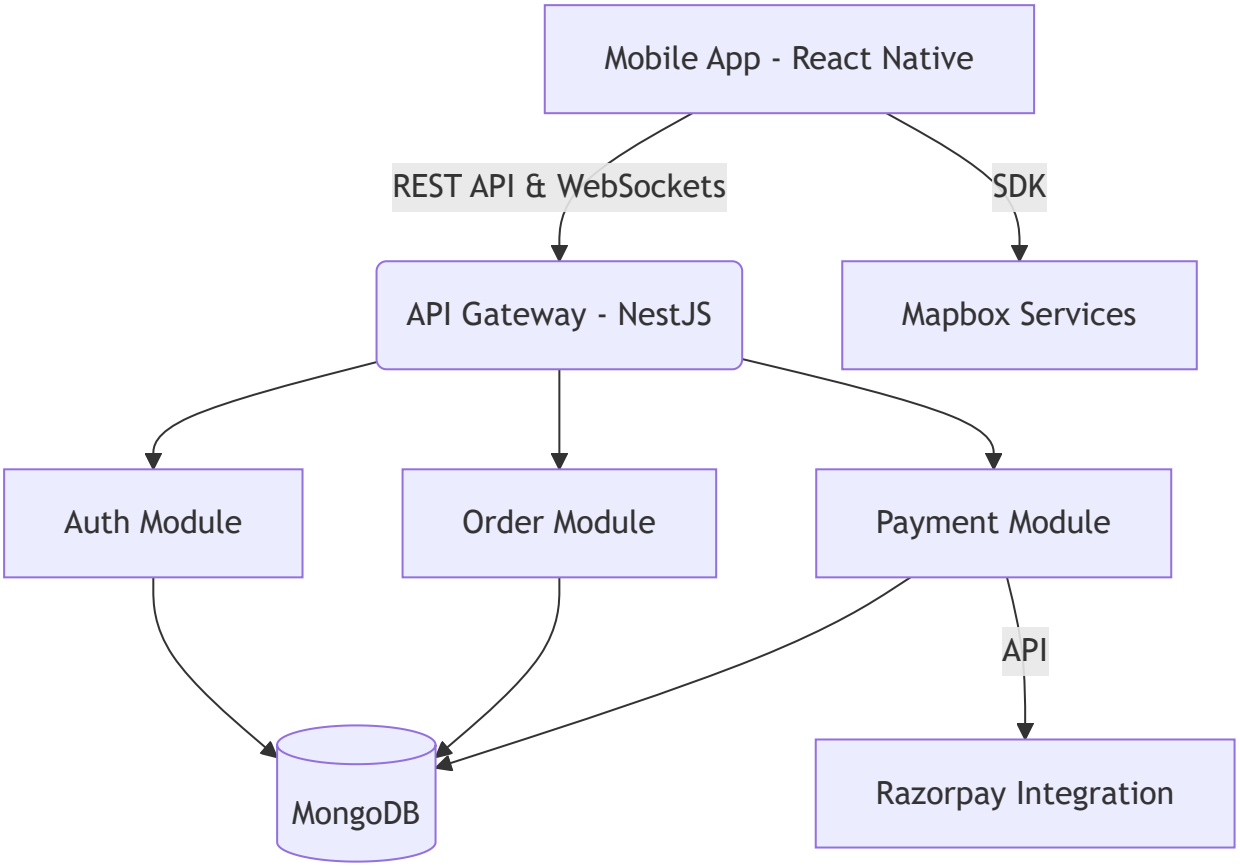


Figure 4.1: High-level System Architecture Diagram

undefined

Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

undefined

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic

signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

4.2 Database Schema

Table 4.1: Database schema definitions for the core operational collections

Collection	Fields	Description
Users	_id, email, role, password	Store user credentials
Restaurants	_id, name, address, isOpen	Store restaurant details
Orders	_id, userId, restaurantId, status, total	Store transaction details

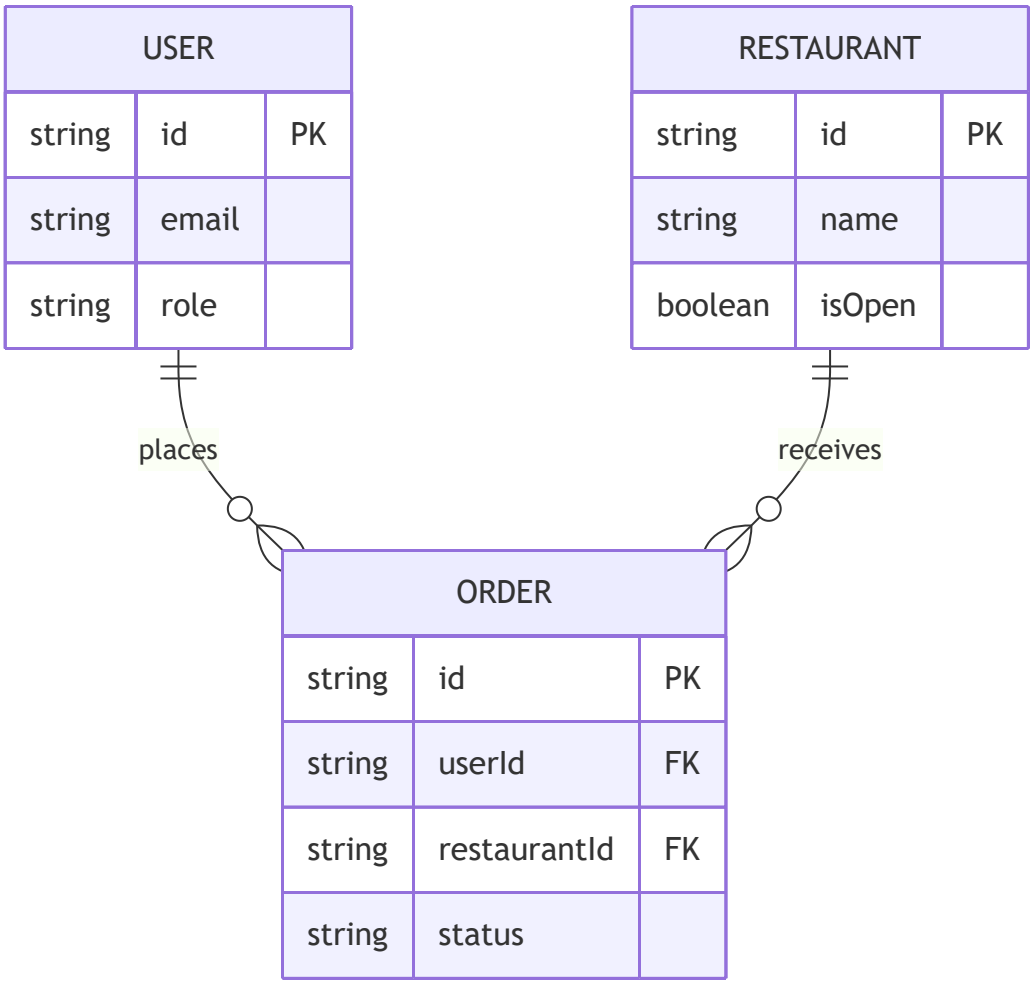


Figure 4.2: Entity-Relationship Diagram for MongoDB Schema

4.3 Data Flow Diagrams

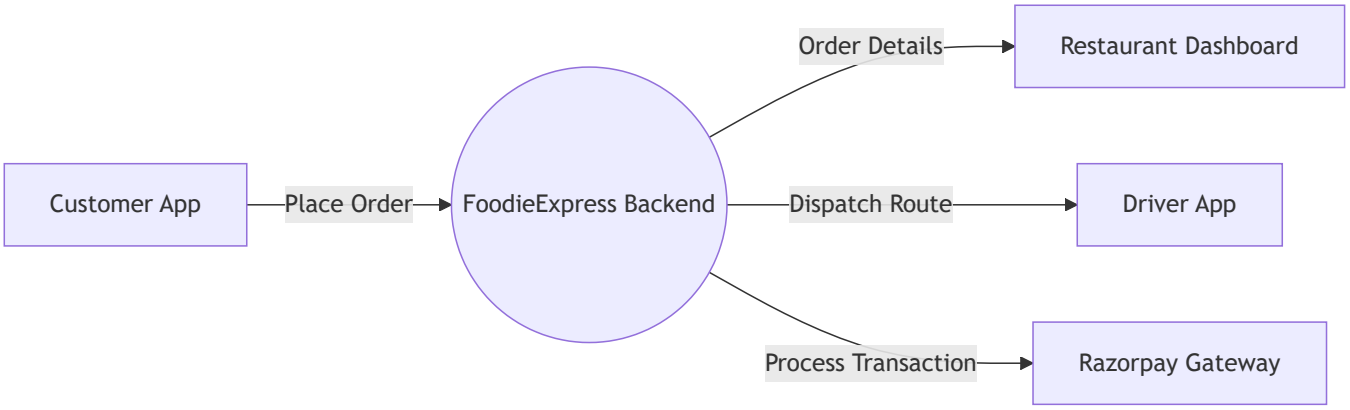


Figure 4.3: Level 0 Data Flow Diagram

undefined

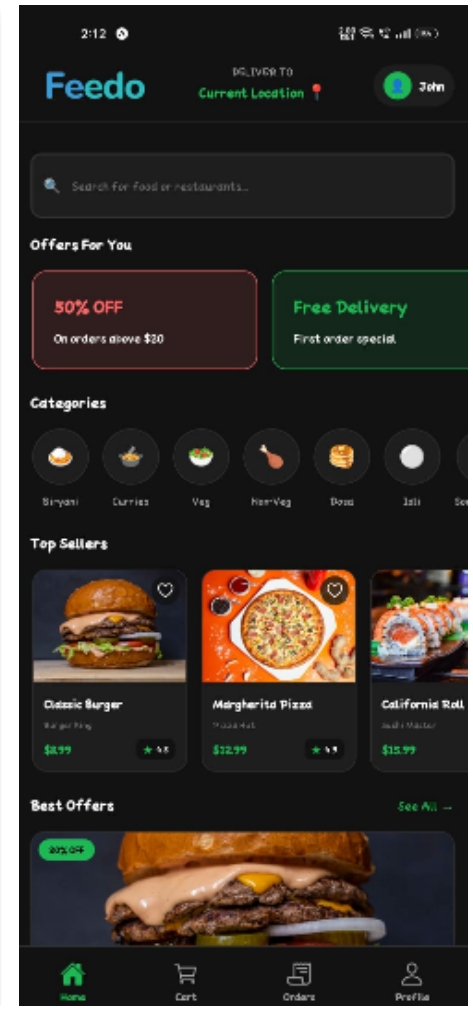
The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

undefined

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

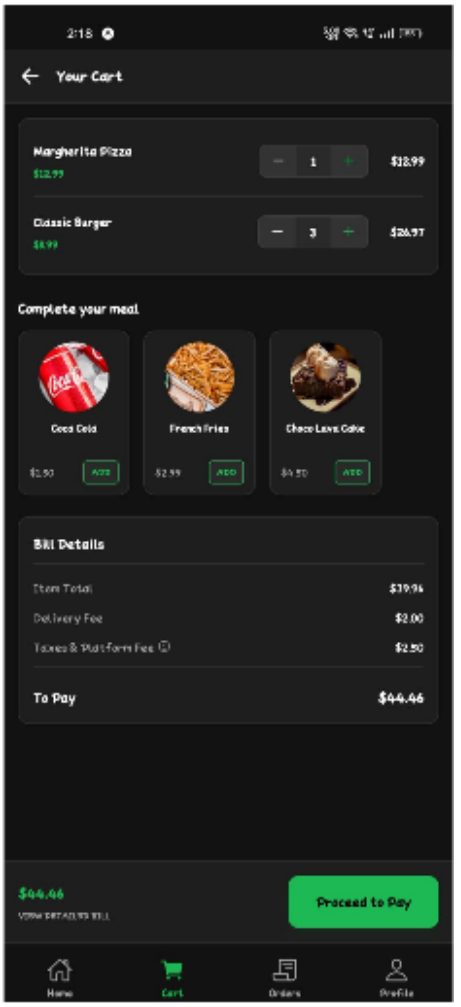
5 Implementation, Testing and Results

5.1 Frontend Implementations



Performance optimization strategies were implemented across the entire stack. On the frontend, React Native was chosen for its ability to deliver native-like performance across both iOS and Android platforms while maintaining a single codebase. Components were meticulously designed to minimize re-renders, and heavy computations were offloaded to background threads where appropriate. Image assets are lazy-loaded and cached aggressively to reduce network latency and data consumption, ensuring the application remains responsive even under constrained network conditions.

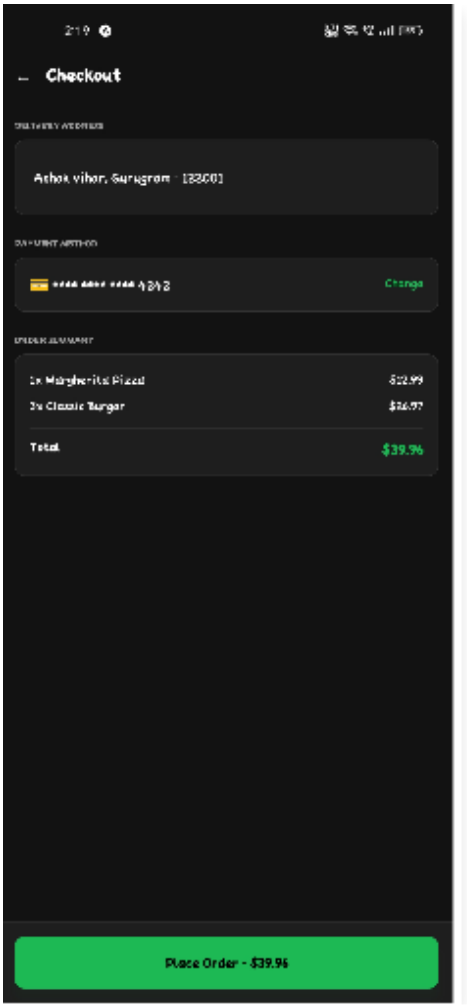
Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.



undefined

undefined

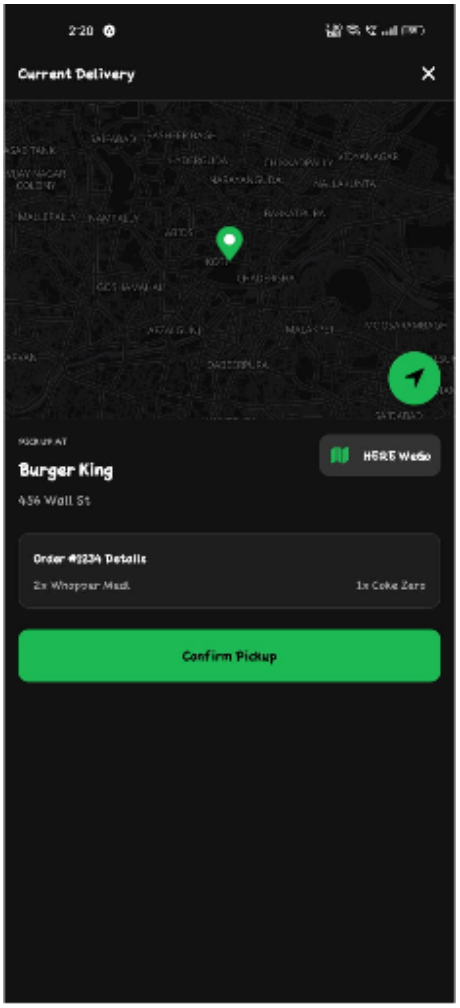
5.2 Payment Integration



Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

The rapid advancement of mobile technology and internet accessibility has catalyzed a significant transformation in the food service industry. Food delivery applications have transitioned from being a mere convenience to an essential utility for urban populations. This paradigm shift necessitates the development of robust, scalable, and secure platforms capable of handling high transaction volumes while ensuring a seamless user experience. The FoodieExpress project addresses these imperatives by leveraging modern software architectural patterns and cutting-edge technologies.

5.3 Live Tracking Implementation



undefined

undefined

5.4 Test Methodologies

Table 5.1: Comprehensive test case suite for the payment verification module

Test ID	Scenario	Expected	Result
TC-PAY-01	Valid Signature	Success	PASS
TC-PAY-02	Invalid Signature	Failure	PASS

undefined

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations

between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

undefined

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

5.5 Performance Evaluation

A comprehensive analysis of user demographics reveals a strong preference for applications that offer intuitive interfaces, diverse restaurant options, and transparent order tracking. Traditional food delivery mechanisms often suffer from latent communication channels, leading to mismatched expectations between customers, restaurants, and delivery personnel. By implementing a real-time event-driven architecture using WebSockets, FoodieExpress ensures that all stakeholders are consistently informed about the order status, thereby mitigating anxiety and enhancing satisfaction.

The architectural design of FoodieExpress employs a microservices-inspired monolithic approach on the backend, utilizing NestJS to provide a highly modular and maintainable codebase. This backend acts as the central nervous system, orchestrating interactions between the client applications and external services. The decision to use MongoDB as the primary data store was driven by its schema flexibility, which accommodates the dynamic nature of restaurant menus and order configurations without the overhead of complex relational migrations.

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

Location-based services are critical for the operational efficiency of the delivery fleet. The integration of Mapbox provides high-fidelity mapping and routing capabilities. Delivery partners are equipped with real-time navigation that optimizes routes based on traffic conditions and delivery constraints. Furthermore, the customer application renders a live map showing the precise location of the assigned driver, updating dynamically as the driver progresses towards the destination. This transparency builds trust and significantly reduces customer support inquiries.

undefined

Security is paramount in e-commerce applications, particularly concerning user data and financial transactions. FoodieExpress integrates Razorpay as its payment gateway, employing a rigorous two-step verification process. Upon initiating a checkout, the backend generates a unique order ID which is transmitted to the client. After the user completes the payment on the Razorpay interface, a cryptographic signature is generated and sent back to the server. The backend validates this signature using an HMAC-SHA256 algorithm with a secret key, ensuring that the transaction is authentic and unaltered.

undefined

Quality assurance processes involved a multi-tiered testing strategy. Unit tests were authored to validate individual business logic components, particularly the pricing algorithms and payment verification routines. Integration tests ensured that the API endpoints interacted correctly with the database and external services. System-level testing, including load testing with simulated concurrent users, verified the platform's ability to scale elastically during peak operational hours without degradation in response times or transaction success rates.

REFERENCES

[1] Facebook Open Source, "React Native: A framework for building native apps using React", 2026. [Online]. Available: https://reactnative.dev/

[2] Kamil Myśliwiec, "NestJS: A progressive Node.js framework for building efficient, reliable and scalable server-side applications", 2026. [Online]. Available: https://nestjs.com/

[3] MongoDB Inc., "MongoDB: The Developer Data Platform", 2026. [Online]. Available: https://www.mongodb.com/

[4] Razorpay Software Private Limited, "Razorpay API Reference: Accept, Process and Disburse Payments", 2026. [Online]. Available: https://razorpay.com/docs/api/

[5] Mapbox, "Mapbox: Maps, Navigation, Search, and Data", 2026. [Online]. Available: https://www.mapbox.com/